Memory-Efficient Versioned File Indexer

CS253 Assignment 1 Report

Name: Arnav Gupta
Roll Number: 240189
Course: CS253

Indian Institute of Technology Kanpur

Contents

1 Introduction and Overview **2**

2 Design and Approach **2**
2.1 Pipeline Architecture 2
2.2 OOP Principles 2
2.3 Carry Buffer for Cross-Boundary Tokens 2
2.4 Template Utility 2

3 Description of Classes **3**
3.1 `FileReader` — Abstract Base Class 3
3.2 `BufferedFileReader` — Derived Class 3
3.3 `Tokenizer` 3
3.4 `VersionedIndexer` 4
3.5 `QueryProcessor` 4

4 Fixed-Size Buffer Processing **4**

5 Example Runs **5**
5.1 Word Count Query 5
5.2 Top-K Query 5
5.3 Difference Query 6

6 Requirements Checklist **6**

7 Assumptions and Observations **6**
7.1 Assumptions 6
7.2 Observations 7

8 Conclusion **7**

1 Introduction and Overview

Large-scale text processing systems routinely handle files that exceed available RAM. Loading such files entirely into memory is impractical. This assignment addresses that by building a **memory-efficient, versioned word-level file indexer** in C++.

Instead of reading the whole file at once, the program reads it in *fixed-size chunks*, tokenises each chunk into words, and accumulates word frequencies in a hash map. Memory usage stays proportional to the number of unique words, not file size.

The program supports three query types:

- `word` — frequency of a word in a given version
- `top` — top-K most frequent words in a version
- `diff` — frequency difference of a word across two versions

2 Design and Approach

2.1 Pipeline Architecture

The program is structured as a linear pipeline:

$$\texttt{File} \xrightarrow{\texttt{BufferedFileReader}} \texttt{Chunk} \xrightarrow{\texttt{Tokenizer}} \texttt{Tokens} \xrightarrow{\texttt{VersionedIndexer}} \texttt{Index} \xrightarrow{\texttt{QueryProcessor}} \texttt{Output}$$

Each stage is a separate class with a single responsibility. The `main` function parses CLI arguments, calls `buildIndex()`, and delegates to `QueryProcessor`.

2.2 OOP Principles

- **Abstraction** — `FileReader` defines the reading interface via a pure virtual method.
- **Inheritance** — `BufferedFileReader` derives from `FileReader`.
- **Encapsulation** — each class exclusively owns its internal state.
- **Single Responsibility** — no class performs more than one logical task.

2.3 Carry Buffer for Cross-Boundary Tokens

Words that straddle a chunk boundary would be silently split without special handling. The `Tokenizer` saves any partial token at the end of a chunk in a `carry` string, which is prepended when the next chunk is processed. A `flush()` call at the end recovers the final token.

2.4 Template Utility

A function template `toLowerCase<T>` normalises strings for case-insensitive matching:

```
template <typename T>
T toLowerCase(T s) {
    for (char &c : s)
        c = tolower(c);
    return s;
}
```

Listing 1: Function template for case normalisation

3 Description of Classes

3.1 `FileReader` — Abstract Base Class

Declares the pure virtual method `readChunk(string &chunk)` and a virtual destructor. All file-reading implementations derive from this class.

```
class FileReader {
public:
    virtual bool readChunk(string &chunk) = 0;
    virtual ~FileReader() {}
};
```

Listing 2: Abstract base class

3.2 `BufferedFileReader` — Derived Class

Inherits from `FileReader` and implements `readChunk()` using `ifstream::read()` with a fixed-size buffer. Throws `std::runtime_error` on file open failure.

```
bool readChunk(string &chunk) override {
    chunk.clear();
    if (file.eof()) return false;
    chunk.resize(bufferSize);
    file.read(&chunk[0], bufferSize);
    chunk.resize(file.gcount());
    return !chunk.empty();
}
```

Listing 3: Buffered chunk reading

The buffer size is fixed at construction (`bufferKB * 1024` bytes) and never resized, ensuring constant per-read memory overhead.

3.3 `Tokenizer`

Iterates over each character in a chunk: alphanumeric characters are lowercased and accumulated; any other character flushes the current token to the output vector. The tail of an incomplete token is preserved in `carry`.

```
vector<string> tokenize(const string &data) {
    vector<string> tokens;
    string current = carry;
    carry.clear();
    for (char c : data) {
        if (isalnum(c)) current += tolower(c);
        else {
            if (!current.empty()) {
                tokens.push_back(current);
                current.clear();
            }
        }
    }
    carry = current;
    return tokens;
}
```

Listing 4: Tokenizer with carry buffer

3.4 VersionedIndexer

Maintains `versions[versionName][word] = count` using nested `unordered_maps`. Exposes three operations:

Method	Description
`addWord(v, w)`	Increments `versions[v][w]` by 1
`getWordCount(v, w)`	Returns the count of word `w` in version `v`
`getTopK(v, k)`	Returns top-k words sorted by descending frequency

3.5 QueryProcessor

Holds a reference to the `VersionedIndexer` and implements the three query methods. For example, the diff query:

```
void diffQuery(const string &v1, const string &v2, const string
   &word) {
    long long c1 = indexer.getWordCount(v1, toLowerCase(word));
    long long c2 = indexer.getWordCount(v2, toLowerCase(word));
    cout << "Difference (v2 - v1): " << (c2 - c1) << endl;
}
```

Listing 5: Diff query

4 Fixed-Size Buffer Processing

The `buildIndex()` function ties the pipeline together:

```
void buildIndex(const string &filePath, const string &version,
```

```
                    size_t bufferKB, VersionedIndexer &indexer) {
    BufferedFileReader reader(filePath, bufferKB);
    Tokenizer tokenizer;
    string chunk;

    while (reader.readChunk(chunk)) {
        auto tokens = tokenizer.tokenize(chunk);
        for (auto &w : tokens)
            indexer.addWord(version, w);
    }

    string last = tokenizer.flush();
    if (!last.empty()) indexer.addWord(version, last);
}
```

Listing 6: Index-building loop

At any point, only one buffer (256–1024 KB) plus the word-frequency map reside in memory. The file is never fully loaded; the loop terminates when `readChunk()` returns `false` at EOF.

$$\text{Memory} \propto \text{buffer size} + |\text{unique words}|$$

5 Example Runs

5.1 Word Count Query

```
./analyzer --file dataset_v1.txt --version v1 \
           --buffer 512 --query word --word error
```

Listing 7: Output

```
Version: v1
Word count: 142
Buffer size: 512 KB
Execution time: 0.38 seconds
```

5.2 Top-K Query

```
./analyzer --file dataset_v1.txt --version v1 \
           --buffer 512 --query top --top 5
```

Listing 8: Output

```
Top-5 words in version v1:
error   -> 142
info    -> 98
debug   -> 75
warning -> 60
fatal   -> 33
```

```
Buffer size: 512 KB
Execution time: 0.41 seconds
```

5.3 Difference Query

```
./analyzer --file1 dataset_v1.txt --version1 v1 \
           --file2 dataset_v2.txt --version2 v2 \
           --buffer 512 --query diff --word error
```

Listing 9: Output

```
Difference (v2 - v1): 27
Buffer size: 512 KB
Execution time: 0.72 seconds
```

6 Requirements Checklist

Requirement	How It Is Met	Section
Abstract base class	`FileReader` with pure virtual `readChunk()`	§3.1
Derived class	`BufferedFileReader` overrides `readChunk()`	§3.2
Runtime polymorphism	Virtual dispatch via `FileReader` pointer	§3.1–3.2
Function template	`toLowerCase<T>`	§2.4
Exception handling	`try/catch` in `main`; `throw` in constructor	`main()`
≥4 classes	FileReader, BufferedFileReader, Tokenizer, VersionedIndexer, QueryProcessor	§3
Fixed buffer	256–1024 KB, constant throughout execution	§4
Case-insensitive	Tokens lowercased at tokenisation time	§3.3

7 Assumptions and Observations

7.1 Assumptions

1. **Word definition.** A word is any maximal contiguous sequence of alphanumeric characters (`isalnum()`). All other characters act as delimiters.

2. **Case folding.** Tokens are lowercased immediately during tokenisation; query words are normalised before lookup.

3. **Version names.** Version identifiers are arbitrary user-supplied strings.

4. **Single query per run.** The `-query` flag selects one query type; multiple queries require separate invocations.

5. **Buffer range.** The program accepts 256–1024 KB as specified; the upper bound is not enforced in code.

6. **Top-K tie-breaking.** Words sharing the same frequency at rank k are ordered by `std::sort`, which is not stable by default.

7.2 Observations

- **Buffer size vs. I/O performance.** Larger buffers reduce system call overhead, but the hash map typically dominates memory for large vocabularies.
- **Hash map complexity.** `unordered_map` gives $O(1)$ average insert and lookup. Sorting for top-K costs $O(V \log V)$ in vocabulary size V, which is typically far smaller than file size.
- **Scalability.** Only the frequency map is retained, so files of arbitrary size can be indexed as long as the vocabulary fits in memory.
- **Extensibility.** A new source type (e.g. compressed or streamed input) only needs to implement `FileReader`; the rest of the pipeline is unaffected.

8 Conclusion

The implementation meets all functional and design requirements. Fixed-size buffering bounds memory use regardless of file size; the object-oriented decomposition keeps components independent and testable. The carry-buffer mechanism in `Tokenizer` is the most critical correctness detail, ensuring words split across chunk boundaries are counted accurately.

Language	C++
Classes	5
Buffer range	256 KB – 1024 KB
Query types	`word`, `top`, `diff`
Memory model	O(unique words), independent of file size